

Assignment 1: Implement Inheritance Types using JavaScript

Introduction: Inheritance is an important object-oriented programming concept. It allows one class to acquire the properties and methods of another class. In JavaScript, class inheritance is commonly implemented using the **extends** keyword. JavaScript does not support multiple inheritance directly with classes, so similar behavior can be achieved using mixins.

1. Single Inheritance

Theory: In single inheritance, one child class inherits the properties and methods of one parent class.

Program:

```
class Animal {      eat() {
  console.log("Animal is eating");    } }

class Dog extends Animal {      bark() {
  console.log("Dog is barking");    } }

let dog = new Dog();

dog.eat();
dog.bark();
```

Output:

Animal is eating

Dog is barking

2. Multilevel Inheritance

Theory: In multilevel inheritance, a class inherits from another class, and another class further inherits from it. This forms a chain of inheritance.

Program:

```
class Grandparent {      house() {
  console.log("Grandparent has a house");    } }

class Parent extends Grandparent {
  car() {      console.log("Parent has a
car");    } }

class Child extends Parent {      bike() {
  console.log("Child has a bike");    } }
let child = new Child();

child.house();
child.car();
child.bike();
```

Output:

Grandparent has a house

Parent has a car

Child has a bike

3. Hierarchical Inheritance

Theory: In hierarchical inheritance, multiple child classes inherit from the same parent class.

Program:

```
class Animal {      eat() {
  console.log("Animal is eating");      } }

class Dog extends Animal {      bark() {
  console.log("Dog is barking");      } }

class Cat extends Animal {      meow() {
  console.log("Cat is meowing");      } }

let dog = new Dog();
let cat = new Cat();

dog.eat();
dog.bark();

cat.eat();
cat.meow();
```

Output:

Animal is eating

Dog is barking

Animal is eating

Cat is meowing

4. Multiple Inheritance

Theory: Multiple inheritance means inheriting features from more than one parent. JavaScript does not support this directly using the extends keyword, but it can be achieved using mixins such as Object.assign().

Program:

```
class Father {      fatherProperty() {
  console.log("Father's property");      } }

class Mother {
  motherProperty() {
    console.log("Mother's property");
  } }

class Child {
}
// Adding methods from both classes

Object.assign(
  Child.prototype,
  Father.prototype,
  Mother.prototype
);

let child = new Child();

child.fatherProperty();
child.motherProperty();
```

Output:

Father's property

Mother's property

5. Hybrid Inheritance

Theory: Hybrid inheritance is a combination of two or more inheritance types. The following example combines multilevel and hierarchical inheritance.

Program:

```
class Grandparent {      showGrandparent() {
  console.log("This is Grandparent");      } }

class Parent extends Grandparent {
  showParent() {
    console.log("This is Parent");      } }

class Child1 extends Parent {
  showChild1() {      console.log("This
is Child 1");      } }

class Child2 extends Parent {
  showChild2() {      console.log("This
is Child 2");      } }

let child1 = new Child1();
let child2 = new Child2();

child1.showGrandparent();
child1.showParent(); child1.showChild1();

child2.showGrandparent();
child2.showParent(); child2.showChild2();
```

Output:

This is Grandparent

This is Parent

This is Child 1

This is Grandparent

This is Parent

This is Child 2

Conclusion: JavaScript supports inheritance primarily through classes and the extends keyword. Single, multilevel, and hierarchical inheritance can be implemented directly, while multiple and hybrid inheritance can be demonstrated using techniques such as mixins and combinations of inheritance structures.